Notebook Build Audit & Execution System

Unified Framework — Construction, Audit & Resource Analysis

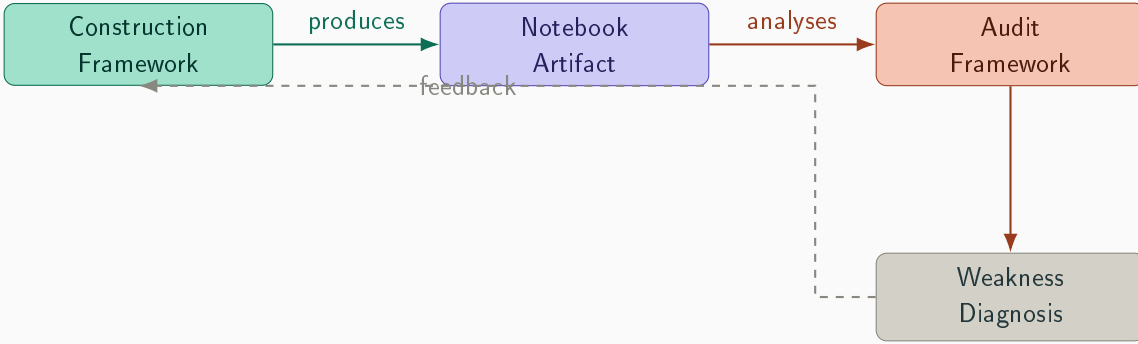
The Challenge

Computational notebooks (Jupyter) are the dominant medium for DS/ML — but they introduce systemic risks:

- Untrusted execution — notebooks from collaborators, open-source, or AI assistants without deterministic safety guarantees
- Lack of deterministic builds — ad hoc environment pinning, missing dependencies, absent seed controls → different results across machines
- No structured audit protocol — no systematic methodology for reviewing notebooks before production, publication, or merge

"Existing code review practices are insufficient for the unique failure modes of notebook-based ML pipelines."

Two Complementary Frameworks



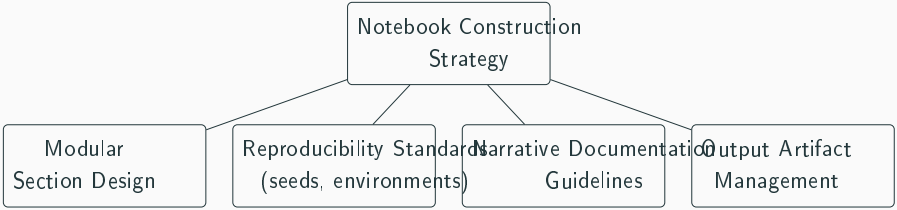
Construction Framework

Forward-looking prescription: guides the author to produce a well-structured, reproducible, and maintainable notebook from the ground up.

Audit Framework

Backward-looking diagnostic: systematically assesses quality, correctness, and risk

Construction Framework — Three Phases



Phase 1 — Scaffold

- Single responsibility
- 8 canonical section headers
- Pin environment upfront
- Global reproducibility controls

Phase 2 — Write

- One section at a time
- Markdown intent per code cell
- Explicit variable passing
- Three-block refactor threshold

Phase 3 — Validate

- Restart kernel per section
- Cell idempotency check
- Linear execution order
- Versioned artifact routing

LLM-as-a-Judge — The Evaluation Paradigm

Why LLM-based evaluation? Notebook auditing combines objective and subjective tasks.

Layer 1 — Deterministic

- Dependency validation
- Execution reproducibility
- Train/test split verification
- Artifact existence

Binary, reproducible — programmatic checks

ROBIN-HOOD [1]:

- Variance-adaptive query allocation
- Same accuracy at half the budget
- Prioritises uncertain passes
- Bound: $\mathcal{O}(\sqrt{\sum \sigma^2 / B})$

Layer 2 — LLM Judge

- Documentation quality
- Conceptual coherence
- Methodological soundness
- Deployment readiness

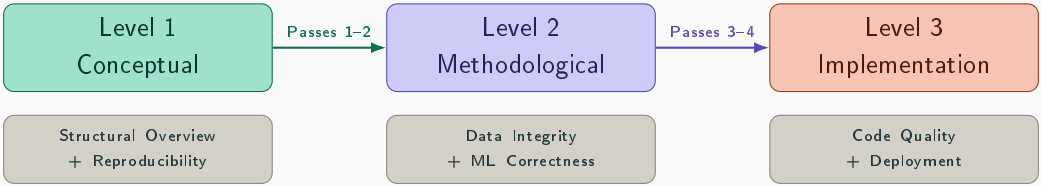
Semantic reasoning — structured evaluation prompts

Metanym Game [2]:

- Generating $\neq$ Evaluating (dissociated)
- Best generators $=$ middling judges
- Panel of diverse judges recommended
- $r = 0.92$ with GPQA Diamond

Key insight: Decompose evaluation into explicit criteria (G-Eval). Known biases: position, verbosity, self-enhancement, dissociation.

Prompt Crafting — Three-Level Structure



Gate 1 → 2

Notebook fails end-to-end or lacks coherent purpose? Decide whether to proceed or flag as blocking.

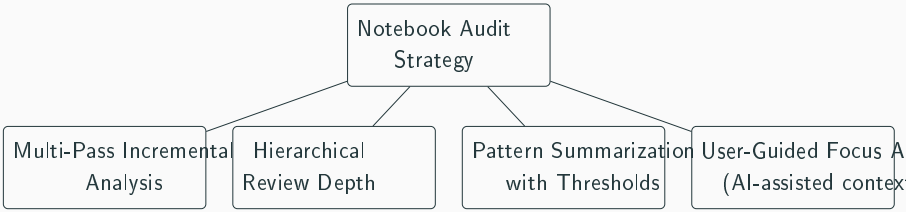
Gate 2 → 3

Methodological flaws (leakage, invalid CV)? Halt or proceed with caveats.

After Level 3

Final report with risk scores across all 6 passes, actionable recommendations.

Six-Pass Audit Protocol



Level 1 — Conceptual

1. **Pass 1** — Structural Overview
2. **Pass 2** — Reproducibility Check

Deliverables: Section map, red flags

Level 2 — Methodological

3. **Pass 3** — Data Integrity Review
4. **Pass 4** — ML Correctness Audit

Deliverables: Pipeline report, checklist

Level 3 — Implementation

5. **Pass 5** — Code Quality Review
6. **Pass 6** — Deployment Readiness

Deliverables: Smell report, readiness score

Architecture Overview

Concept

- Construction ↔ Audit lifecycle
- Three-level audit with HITL gates
- Provider-agnostic LLM layer

Physical

- Docker Compose (5 services)
- Ephemeral sandbox per notebook audit
- Local or cloud (Ollama/OpenAI)

Logical

- React + TypeScript Dashboard
- FastAPI REST + SSE Gateway
- LLM Orchestrator (Strategy Pattern)
- Audit State Machine
- @jupyter-kit Execution Engine
- PostgreSQL + SQLAlchemy

Resource Costs

Computational

Item	USD
Server PC	$1,000
Hosting (3mo)	$300
Total Q1	**$1,300**

Development: 8 weeks, 2 developers
Total: $5,000 USD

Energetic — Third Pillar Proposal

- LLM inference is energy-intensive (0.01–0.1 kWh per pass)
- Notebook execution often wasteful
- ESG reporting increasingly required

Proposed compromise:

- **Pass 7** (optional) — Energy Impact Assessment
- Backends: RAPL, NVML, powermetrics
- Graceful degradation when unavailable

Timeline & Roadmap

Phase	Summary	Duration
1	Dockerization, monorepo, sandbox spawning, dependency pinning	2 weeks
2	FastAPI REST + SSE, LLM Provider strategy, Audit State Machine, @jupyter-kit integration	3 weeks
3	React dashboard, session management, live progress, risk visualisation	2 weeks
4	E2E testing, prompt validation, security review, documentation	1 week

Total: 8 weeks — 2 developers

What do we build first?

Provider Strategy

- Local (Ollama) vs cloud (OpenAI) as default?
- Offline-first or hybrid?

Pass 7 Energy

- Include from Phase 2 as experimental?
- Defer to Phase 4?

Scope

- Full 6-pass from day one?
- Start with Level 1 only?

Audience

- Internal team only, or client-ready?
- Academic publication use case?

Notebook Build Audit & Execution System

Construction

Build notebooks right from scratch.

3 phases, 8 canonical sections.

Audit

Systematically review any notebook.

6 passes, 3 levels, HITL gates.

Resource

(Proposed) Track energy & compute.

Optional Pass 7.

Codificar experiencia humana, validar con supervisión sistemática.

References i

- A. Saha, A. Wagde, and B. Kveton, "LLM-as-Judge on a Budget," *arXiv:2602.15481*, 2026.
- D. Nordfors, "The Metanym Game: A Self-Contained, Self-Consistent LLM Peer-Community Benchmark for Structural Intelligence," *arXiv:2606.21008*, 2026.